

Estrutura de Dados — Revisão para a A1

Simulado — não vale nota

Disciplina: Estrutura de Dados — BSI — UNITINS **Professor:** Alysson M. Bruno **Conteúdo:** Aulas 01 a 07 **Prova A1:** 09/10/2026

Como fazer este simulado

- Faça **sem consulta**: feche o texto de revisão, os slides e o caderno. O objetivo é descobrir o que você já sabe fazer sozinho, como vai ser na prova.
- Separe **cerca de 45 minutos** sem interrupção. Treine o ritmo: na prova, o tempo também conta.
- Nas objetivas, anote a letra escolhida na tabela de respostas. Cada questão tem quatro alternativas (A, B, C e D) e apenas uma está correta.
- Nas questões com código, **acompanhe a execução no papel**, linha por linha, anotando o valor das variáveis. Não chute pelo “jeitão” do código.
- A discursiva (Questão 11) deve ser **escrita à mão, no papel**, com caneta. A prova é manuscrita: quem só treinou digitando descobre na hora da prova que esquece o ponto e vírgula, a chave que fecha o bloco e o return. Escrever à mão agora é o treino certo.
- Só depois de terminar tudo, abra o capítulo **“Resolução comentada do simulado”**, no fim do texto de revisão, e corrija você mesmo.

Quadro de respostas (Parte I)

Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10

Parte I — Questões Objetivas (10 × 0,5 = 5,0 pontos)

Questão 1

Uma academia de ginástica organizou o programa de treinos assim: o arquivo fontes/br/unitins/academia/Treino.java começa com package br.unitins.academia; e contém o método main. Na raiz do projeto, a equipe compilou com javac -d saida fontes/br/unitins/academia/Treino.java e, em seguida, executou o programa com sucesso usando java -cp saida br.unitins.academia.Treino.

Considerando as informações apresentadas, assinale o caminho em que o compilador gravou o arquivo Treino.class.

A saida/Treino.class

B fontes/br/unitins/academia/Treino.class

C br/unitins/academia/saida/Treino.class

D saida/br/unitins/academia/Treino.class

Questão 2

A mesma academia escreveu as classes abaixo para calcular as mensalidades.

```
public interface Plano { int mensalidade(); }

public abstract class Aluno implements Plano {
    protected String nome;
    public String getNome() { return nome; }
}

public class AlunoMensal extends Aluno {
    public AlunoMensal(String nome) { this.nome = nome; }
    @Override
    public int mensalidade() { return 120; }
}

public class AlunoAnual extends Aluno {
    public AlunoAnual(String nome) { this.nome = nome; }
    @Override
    public int mensalidade() { return 90; }
}
```

No método main, executa-se:

```
Aluno[] turma = { new AlunoMensal("Lia"), new AlunoAnual("Davi"),
                  new AlunoMensal("Nara") };
int total = 0;
for (Aluno a : turma) {
    total += a.mensalidade();
}
System.out.println(total);
```

Considerando o código apresentado, avalie as afirmações a seguir.

I. O programa imprime 330.

II. A declaração `Aluno[] turma` compila, mesmo sendo `Aluno` uma classe abstrata, porque a regra proíbe criar um objeto com `new Aluno()`, e não declarar variáveis desse tipo.

III. Em `a.mensalidade()`, quem decide a versão executada é o tipo declarado `Aluno`; por isso os três alunos pagam o mesmo valor.

IV. Se o construtor de `AlunoAnual` fosse escrito como `nome = nome;`, o programa deixaria de compilar.

É correto apenas o que se afirma em

A I e III.

B I e II.

C II, III e IV.

D I, II e IV.

Questão 3

O sistema de uma feira livre executa o trecho abaixo sobre um vetor `v` com `n` preços.

```
int maior = v[n - 1];
for (int i = n - 1; i >= n / 2; i--) {    // só a metade de cima do vetor
    if (v[i] > maior) maior = v[i];
}
```

Em um teste com `n = 2.500`, o trecho levou 5 s. A equipe vai passar a usar `n = 7.500` no mesmo computador.

Considerando que o tempo cresce como o número de operações do trecho, assinale a melhor estimativa.

A Cerca de 15 s, porque o trecho é $O(n)$: o laço dá $n/2$ voltas, e $n/2$ também fica 3 vezes maior.

B Cerca de 7,5 s, porque o laço percorre só metade do vetor, e isso divide por 2 o fator 3.

C Cerca de 45 s, porque o trecho é $O(n^2)$: multiplicar `n` por 3 multiplica o trabalho por 9.

D Cerca de 5 s, porque o laço anda para trás e para no meio, e por isso o número de voltas não depende de `n`.

Questão 4

Para entender a pilha de chamadas, um estudante escreveu o método recursivo abaixo, que soma os algarismos de um número inteiro positivo.

```
static int somaDigitos(int n) {
    if (n < 10) {
        System.out.print(n + " ");
        return n;
    }
    int r = n % 10 + somaDigitos(n / 10);
    System.out.print(r + " ");
    return r;
}
```

O programa chama `somaDigitos(472)`. Lembre-se: $472 \% 10$ vale 2 e $472 / 10$ vale 47 (divisão inteira).

Considerando a ordem em que as chamadas entram e saem da pilha, assinale a alternativa que mostra exatamente o que é impresso.

A 13 11 4

B 2 9 13

C 4 11 13

D 472 47 4

Questão 5

Uma lanchonete guarda os pedidos do dia no arquivo `pedidos.txt`, com seis linhas, um pedido por linha e nenhuma linha em branco. Para mostrar os pedidos, um estagiário escreveu:

```
try (BufferedReader leitor = new BufferedReader(
    new FileReader("pedidos.txt", StandardCharsets.UTF_8))) {
    while (leitor.readLine() != null) {
        System.out.println(leitor.readLine());
    }
}
```

Considerando o contexto apresentado, avalie as asserções a seguir e a relação proposta entre elas.

I. O programa mostra apenas o 2º, o 4º e o 6º pedido do arquivo.

PORQUE

II. Cada chamada de `readLine()` consome uma linha: a chamada do `while` gasta uma linha só no teste, e a chamada do corpo lê a linha seguinte.

A respeito dessas asserções, assinale a opção correta.

A I e II são verdadeiras, e II explica I: como cada volta do laço chama `readLine()` duas vezes, metade das linhas é lida e descartada no teste.

B I e II são verdadeiras, mas II não explica I: os pedidos de posição ímpar somem porque o `try-with-resources` fecha e reabre o arquivo a cada volta.

C I é falsa e II é verdadeira: o programa mostra os seis pedidos, porque a chamada do `while` e a do corpo leem a mesma linha.

D I é verdadeira e II é falsa: `readLine()` só avança para a próxima linha quando o valor devolvido é guardado em uma variável.

Questão 6

O aplicativo de edição de fotos de um estúdio guarda, em uma sequência, os filtros aplicados a uma imagem. Cada filtro novo entra no fim; o botão “Desfazer” retira sempre o último filtro; e a tela de detalhes mostra o filtro de uma posição informada (por exemplo, o 3º). A quantidade de filtros varia de imagem para imagem.

A equipe precisa escolher a estrutura de dados, entre as estudadas na disciplina: array de tamanho fixo, `ArrayList` (vetor dinâmico), lista encadeada simples e lista duplamente encadeada.

Considerando as operações mais frequentes do cenário, assinale a decisão mais adequada e a justificativa correta.

A Lista simples com cabeça e cauda, porque inserir no fim e retirar o último custam $O(1)$ graças à referência cauda.

B Array de tamanho fixo, porque o acesso pela posição é $O(1)$ e o array aumenta de tamanho quando fica cheio.

C Lista dupla, porque só ela permite retirar o último elemento da sequência.

D `ArrayList`, porque inserir no fim é $O(1)$ amortizado, retirar o último não desloca nenhum elemento e o acesso pela posição é $O(1)$.

Questão 7

Um estudante reescreveu o método `remove` da classe `ListaEncadeada<T>` da Aula 05, que usa os campos `cabeca`, `cauda` e `tamanho` e o nó `No<T>`, com `dado` e `proximo`. Os demais métodos são iguais aos da aula; `imprimir()` percorre os nós a partir de `cabeca` e mostra, por exemplo, `A → B → null` [`tamanho=2`].

```

public boolean remover(T dado) {
    No<T> ant = cabeca;
    while (ant.proximo != null) {
        if (ant.proximo.dado.equals(dado)) {
            if (ant.proximo == cauda) cauda = ant;
            ant.proximo = ant.proximo.proximo;
            tamanho--;
            return true;
        }
        ant = ant.proximo;
    }
    return false;
}

```

Em seguida, executou:

```

ListaEncadeada<String> lista = new ListaEncadeada<>();
lista.inserirNoFim("Lia");
lista.inserirNoFim("Davi");
lista.inserirNoFim("Nara");
System.out.println(lista.remover("Lia"));
System.out.println(lista.remover("Nara"));
lista.imprimir();

```

Considerando o código apresentado, assinale a alternativa que descreve corretamente o resultado.

- A Imprime true, true e Davi → null [tamanho=1], pois os dois nomes estavam na lista.
- B Imprime false, true e Lia → Davi → null [tamanho=2]: o laço sempre compara ant.proximo, e por isso o dado do nó da cabeça nunca é comparado.
- C Imprime false, false e Lia → Davi → Nara → null [tamanho=3], pois o laço para antes de chegar ao último nó.
- D Lança NullPointerException na primeira chamada, porque ant começa apontando para cabeca.

Questão 8

O painel de uma oficina mecânica chama os carros pela ordem de chegada. Ele usa a fila com array circular da Aula 06, com capacidade 5; os campos inicio, fim e tam começam em 0, e os carros são identificados por números.

```

public void enqueue(int v) {
    if (tam == dados.length) throw new IllegalStateException("Fila cheia.");
    dados[fim] = v;
    fim = (fim + 1) % dados.length;
    tam++;
}

public int dequeue() {
    if (vazia()) throw new IllegalStateException("Fila vazia.");
    int v = dados[inicio];
    inicio = (inicio + 1) % dados.length;
    tam--;
    return v;
}

```

A partir da fila vazia, executam-se, nesta ordem: enqueue(11), enqueue(22), enqueue(33), enqueue(44), dequeue(), dequeue(), enqueue(55), enqueue(66), enqueue(77) e dequeue().

Com base no código e na sequência, avalie as afirmações a seguir.

- I. O último dequeue() devolve 33.
- II. Ao final, fim vale 7.
- III. O valor 66 foi gravado na mesma posição do array em que antes estava o 11.
- IV. Ao final, a fila está cheia: tam vale 5.

É correto apenas o que se afirma em

- A I, II e III.
- B II e IV.
- C I e III.
- D III e IV.

Questão 9

Uma lavanderia ordena, toda noite, as 6 etiquetas das peças do dia pelo número. Quase sempre as etiquetas já chegam em ordem. O sistema usa o Selection Sort da Aula 07, e um estagiário propôs trocá-lo pelo Insertion Sort da mesma aula. Para decidir, a equipe analisou os dois algoritmos no vetor {1, 2, 3, 4, 5, 6}, contando as comparações entre elementos e as trocas ou deslocamentos.

Considerando o que foi estudado sobre Bubble Sort, Selection Sort e Insertion Sort, assinale a decisão mais adequada e a justificativa correta.

A Tanto faz trocar, porque os dois fazem 15 comparações e nenhuma troca: ambos são $O(n^2)$, e a ordem da entrada não muda o trabalho.

B Não trocar, porque o Insertion Sort faz 15 deslocamentos no vetor já ordenado, um para cada par de elementos, enquanto o Selection Sort faz só 5 trocas.

C Tanto faz trocar, porque o Selection Sort também percebe, na primeira passagem, que o vetor está ordenado e para, como o Insertion Sort.

D Trocar, porque no vetor já ordenado o Selection Sort faz 15 comparações e nenhuma troca, enquanto o Insertion Sort faz 5 comparações e nenhum deslocamento: o while para no primeiro teste de cada volta.

Questão 10

Para conferir o Bubble Sort da Aula 07 antes de usá-lo no placar de um campeonato de vôlei, uma estudante aplicou o método abaixo ao vetor $v = \{9, 4, 7, 2, 6\}$ e anotou o estado do vetor ao fim de cada passagem do laço externo.

```
public static void bubbleSort(int[] v) {
    int n = v.length;
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1 - i; j++) {
            if (v[j] > v[j + 1]) {
                int tmp = v[j]; v[j] = v[j + 1]; v[j + 1] = tmp;
            }
        }
    }
}
```

Considerando o contexto apresentado, avalie as asserções a seguir e a relação proposta entre elas.

I. Ao fim da segunda passagem ($i = 1$), o vetor está $[4, 2, 6, 7, 9]$.

PORQUE

II. Em cada passagem, o Bubble Sort leva o menor valor do trecho ainda não ordenado para a primeira posição desse trecho, como faz o Selection Sort.

A respeito dessas asserções, assinale a opção correta.

A I é verdadeira e II é falsa: cada passagem leva o maior valor do trecho para o fim; o 2 anda no máximo uma posição para a esquerda por passagem.

B I e II são verdadeiras, e II explica I: é a ida do menor valor para a frente que produz $[4, 2, 6, 7, 9]$.

C I é falsa e II é verdadeira: ao fim da segunda passagem o vetor já está [2, 4, 6, 7, 9], com o menor valor na primeira posição.

D I e II são falsas: ao fim da segunda passagem o vetor está [4, 7, 2, 6, 9], e o Bubble Sort compara elementos distantes, e não vizinhos.

Parte II — Questão Discursiva (5,0 pontos)

Discursiva (Questão 11)

Uma horta comunitária guarda a ordem de rega dos canteiros em uma lista encadeada simples, como a da Aula 05, com referências para o primeiro nó (cabeca) e para o último (cauda). Os métodos `inserirNoInicio`, `inserirNoFim`, `contem` e `remover` já existem e são iguais aos da aula.

```
public class No<T> {
    T dado;
    No<T> proximo;
    public No(T dado) { this.dado = dado; }
}

public class ListaEncadeada<T> {
    private No<T> cabeca;    // primeiro nó; null se a lista está vazia
    private No<T> cauda;     // último nó; null se a lista está vazia
    private int tamanho;     // quantos nós existem

    // escreva aqui o método obter
}
```

- Complete a classe escrevendo, em Java, o método `T obter(int indice)`, que devolve o dado guardado no nó da posição `indice` (a posição 0 é a da cabeca). Se o índice for inválido — negativo, ou maior ou igual ao número de nós, o que inclui qualquer índice quando a lista está vazia —, o método lança `IndexOutOfBoundsException`. O método não altera a lista.
- Indique a complexidade do método em notação O no pior caso, considerando n o número de nós da lista, e justifique dizendo quantas vezes o trecho mais executado roda. Diga também qual é o melhor caso.

Critérios de correção

- Identificação do problema: descreve o que o método deve fazer e os casos a tratar (índice válido, índice inválido e lista vazia) — 1,0 pt
- Formulação: assinatura correta e uso adequado dos campos e das classes dadas — 0,5 pt
- Desenvolvimento: lógica principal correta (percorre os nós a partir da cabeca, avança o número certo de vezes e devolve o dado) — 2,0 pt

- Interpretação: indica a complexidade $O(n)$ e a justifica pelo número de passos em função de n — 1,0 pt
- Clareza: código organizado, nomes claros e justificativa objetiva — 0,5 pt

Resposta (escreva à mão, em uma folha de papel):

[illegible]

Terminou? Agora abra o texto de revisão no capítulo “**Resolução comentada do simulado**” e corrija as suas respostas, questão por questão, antes de voltar às etapas em que errou.